Because the default behavior is to use all columns in the table as features except the primary keys, and we only want some of the features from those tables, we want to include the `feature_names` argument for the `oil_10d_lag_ft` and `stores_ft` tables. For example, in the oil price feature table, there is an additional date column that we do not want to include in the modeling training set.

```
model_feature_lookups = [
    FeatureLookup(
      table_name="oil_10d_lag_ft",
      lookup_key="date",
      feature_names="lag10_oil_price"
    ),
    FeatureLookup(
      table_name="store_holidays_ft",
      lookup_key=["date","store_nbr"]
    ),
    FeatureLookup(
      table_name="stores_ft",
      lookup_key="store_nbr",
      feature_names=["store_cluster","store_type"]
    ),
]
```

Figure 6.8 – The list of feature lookups for the Favorita Sales Forecasting model

Combine the feature lookups with the raw training data using `create_training_set`. The `training_set` specification can be reused to create a fresh DataFrame of training data with the same logic.

```
training_set = fe.create_training_set(
    df=raw_data,
    feature_lookups=model_feature_lookups,
    label=label_name,
)
training_df = training_set.load_df()
```

Figure 6.9 – Create the training set specification using training data as df, the feature lookups, and the column name of the label

To populate a DataFrame with the training data, call the `load_df()` method. A DataFrame is returned with the current version of features and training data for modeling. For this reason, saving the spec details in a util file can be advantageous.